



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Jaustrylle
July 9, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
 - Data exploration and categorical comparison
 - Python data visualization libraries to analyze relationships between discrete and continuous variables
 - Success metric evaluation
 - SQL and data manipulation concepts using Jupyter Notebooks
 - Writing SQL queries, working with databases, and integrating SQL with data science
 - Booster lifecycle and version isolation using engineering model iterations
- Summary of all results
 - Chronological maturity of booster technology
 - Site performance and workflow distribution centered around Kennedy Space Center
 - Orbit optimization around VLEO and GTO
 - Booster evaluation and weight thresholds, especially from Full Thrust and light payloads

Introduction

- Project background and context
 - During the commercial space age, space travel will be more affordable, and Virgin Galactic is providing suborbital spaceflights to the public. SpaceX has already sent spacecraft to the International Space Station, and Starlink, a satellite internet constellation that provides satellite Internet access, has sent manned missions to space. SpaceX can do this, because rocket launches are relatively inexpensive when compared to the past, and SpaceX advertises Falcon 9 rocket launches that cost 62 million dollars on its website, while other providers charge up to 165 million dollars each, so SpaceX can reuse the first stage with those savings. If we can determine if the first stage will land, we can thus determine how much a launch will cost.
- Problems you want to find answers
 - For this project, SpaceY would like to compete with SpaceX and is founded by billionaire industrialist Allon Musk. We will determine the price of each launch by gathering information about SpaceX and creating dashboards for the team. We will additionally determine if SpaceX will reuse the first stage, but instead of using rocket science to determine if the first stage will land successfully, we will train a machine learning model and use public information to predict this.

Section 1

Methodology

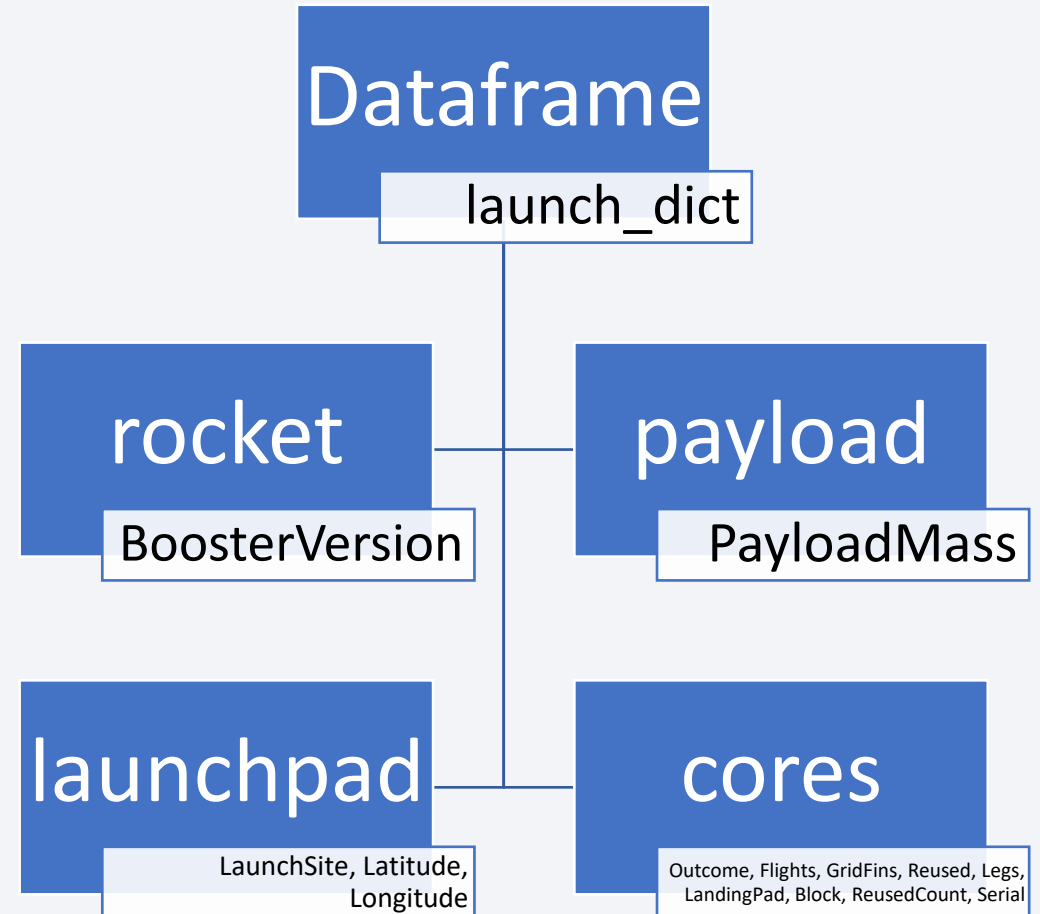
Methodology

Executive Summary

- Data collection methodology:
 - REST API integration fetched publicly available SpaceX data, including flight data, payload metrics, and landing configurations.; web scraping and database storage after
- Perform data wrangling
 - Target variable encoding, missing value imputation (mean), and feature normalization via Standard Scaler
- Perform exploratory data analysis (EDA) using visualization and SQL
 - SQL querying, visual scatter and distribution mapping
- Perform interactive visual analytics using Folium and Plotly Dash
 - Geospatial mapping, dashboard deployment via cloud-based server
- Perform predictive analysis using classification models
 - Model training, hyperparameter tuning (GridSearchCV), evaluation and comparison

Data Collection

- Describe how data sets were collected.
 - API server down, so data was downloaded, parsed, and appended to lists
 - JSON URL used in static response object to make results more consistent
 - Response decoded using JSON normalization and stored into data frame
 - Functions applied globally to following variables



Data Collection – SpaceX API

- Data downloaded from SpaceX REST
- Data filtered to include only Falcon 9 launches
- FlightNumber column reset
- Missing values for PayloadMass imputed using mean
- Result exportable to CSV
- GitHub URL:
<https://github.com/jaustrylle/applied-data-science-capstone/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>

	rocket	payloads	launchpad	cores	flight_number	date_utc	date
0	5e9d0d95eda69955f709d1eb	5eb0e4b5b6c3bb0006eeb1e1	5e9e4502f5090995de566f86	{'core': '5e9e289df35918033d3b2623', 'flight': 1, 'gridfins': False, 'legs': False, 'reused': False, 'landing_attempt': False, 'landing_success': None, 'landing_type': None, 'landpad': None}	1	2006-03-24T22:30:00.000Z	2006-03-24
1	5e9d0d95eda69955f709d1eb	5eb0e4b6b6c3bb0006eeb1e2	5e9e4502f5090995de566f86	{'core': '5e9e289ef35918416a3b2624', 'flight': 1, 'gridfins': False, 'legs': False, 'reused': False, 'landing_attempt': False, 'landing_success': None, 'landing_type': None, 'landpad': None}	2	2007-03-21T01:10:00.000Z	2007-03-21
3	5e9d0d95eda69955f709d1eb	5eb0e4b7b6c3bb0006eeb1e5	5e9e4502f5090995de566f86	{'core': '5e9e289ef3591855dc3b2626', 'flight': 1, 'gridfins': False, 'legs': False, 'reused': False, 'landing_attempt': False, 'landing_success': None, 'landing_type': None, 'landpad': None}	4	2008-09-28T23:15:00.000Z	2008-09-28
				{'core': '5e9e289ef359184f103b2627',			

Data Collection - Scraping

- Webscraping via BeautifulSoup from this link:
https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches
- Column and variable names extracted from HTML header and parsed
- Exportable to CSV
- GitHub URL:
<https://github.com/jaustrylle/applied-data-science-capstone/blob/main/jupyter-labs-webscraping.ipynb>

```
# Orbit
# TODO: Append the orbit into launch_dict with key `Orbit`
orbit = row[5].a.string if row[5].a else row[5].text.strip()
launch_dict["Orbit"] = orbit
print(orbit)

# Customer
# TODO: Append the customer into launch_dict with key `Customer`
customer = row[6].a.string if row[6].a else row[6].text.strip()
launch_dict["Customer"] = customer
print(customer)

# Launch outcome
# TODO: Append the launch_outcome into launch_dict with key `Launch outcome`
launch_outcome = list(row[7].strings)[0].strip()
launch_dict["Launch outcome"] = launch_outcome
print(launch_outcome)

# Booster Landing
# TODO: Append the launch_outcome into launch_dict with key `Booster Landing`
booster_landing = landing_status(row[8])
launch_dict["Booster landing"] = booster_landing
print(booster_landing)
```

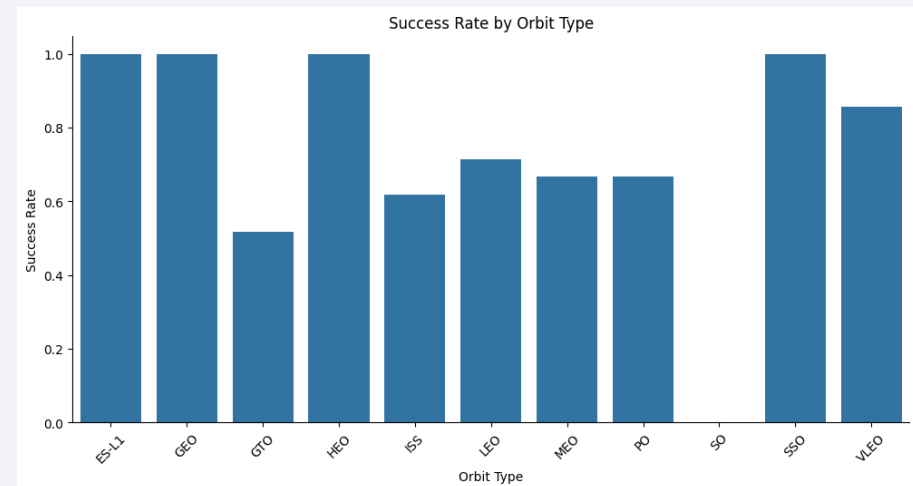
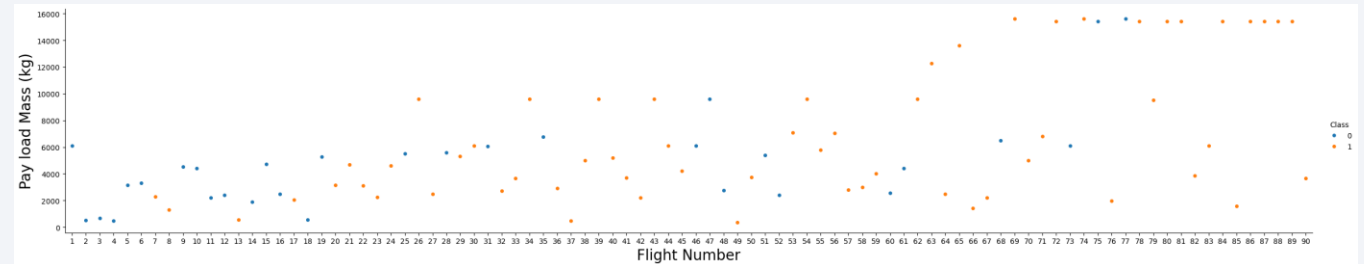
Data Wrangling

- Landing outcomes converted to numeric, binary training labels
- You need to present your data wrangling process using key phrases and flowcharts
- Each launch aims for a dedicated orbit:
 - GTO, ISS, VLEO, PO, LEO, SSO, MEO, ES-L1, HEO, SO, and GEO
 - All outcomes categorized into “bad” or “good” based on whether landing was successful or not
- GitHub URL: <https://github.com/jaustrylle/applied-data-science-capstone/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Lon
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.5
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.5
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.5
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.4
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.5

EDA with Data Visualization

- Key points:
 - PayloadMass vs. FlightNumber -> steady, even climbing trend
 - Success Rate by Orbit Type -> SO needs more testing
 - Features engineered to create dummy variables for categorical columns
- GitHub URL:
<https://github.com/jaustrylle/applied-data-science-capstone/blob/main/edadataviz.ipynb>



EDA with SQL

- Queries to find key trends and calculate statistics, i.e.:

- No attempt to land most common
- 100 successful missions

- GitHub URL:
https://github.com/jaustryle/applied-data-science-capstone/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Average_Payload_Mass
2928.4

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

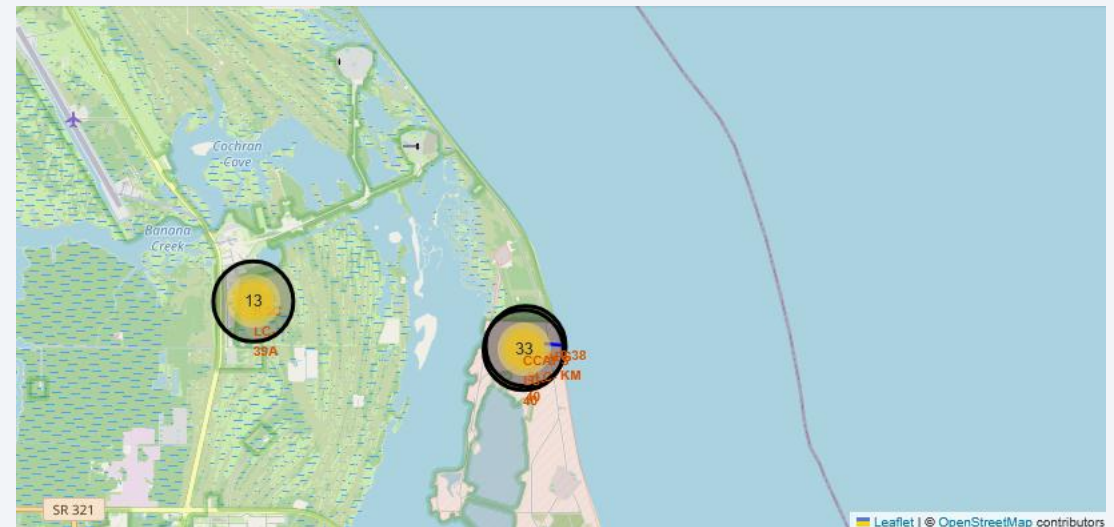
Four Unique
Launch Sites

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

First_Successful_Ground_Pad_Landing_Date
2015-12-22

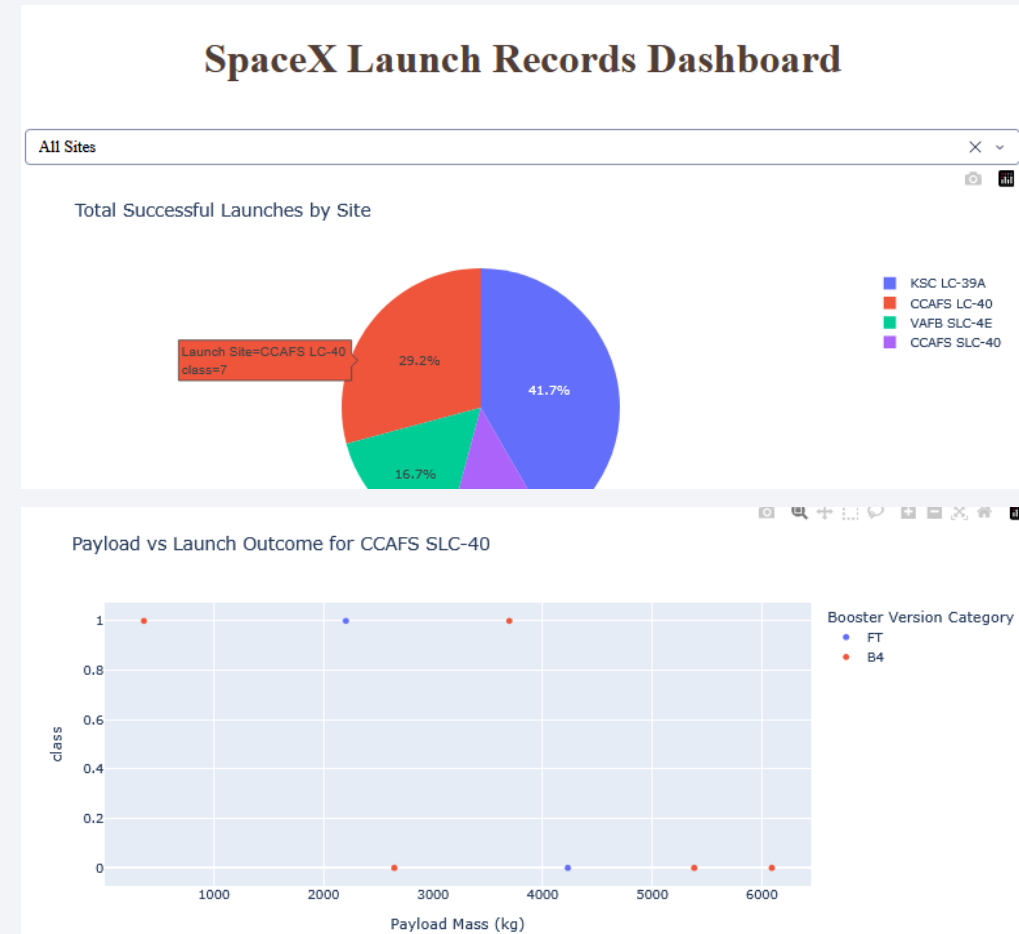
Build an Interactive Map with Folium

- Marked all launch sites on map
- Marked success/failed launches for each site
- Calculated distances between launch size and proximities
- GitHub URL:
https://github.com/jaustrylle/applied-data-science-capstone/blob/main/lab_jupyter_launch_site_location.ipynb



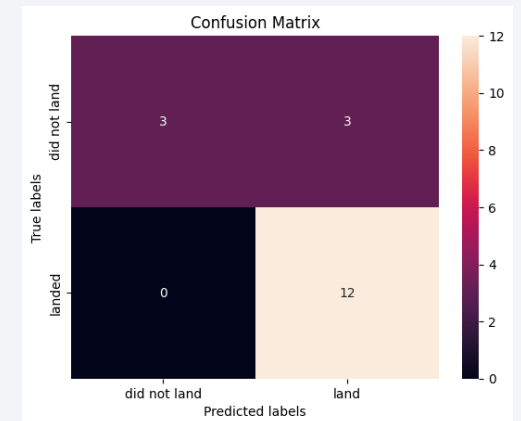
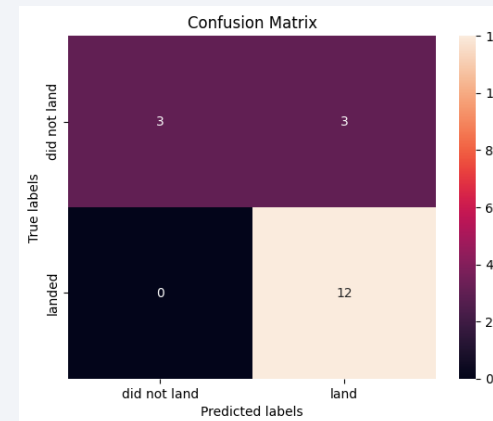
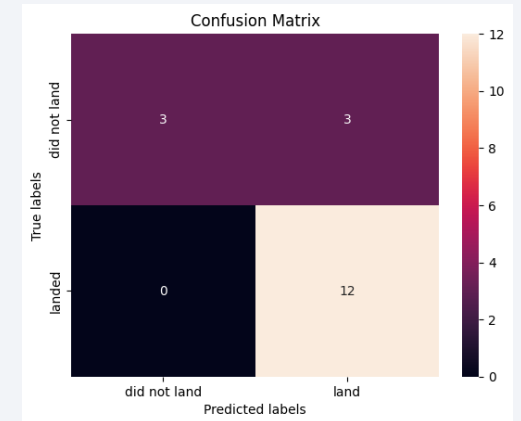
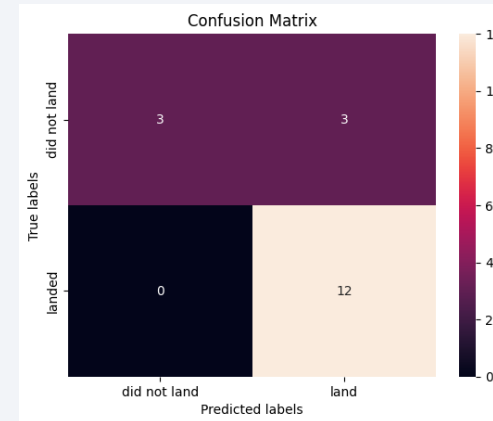
Build a Dashboard with Plotly Dash

- Controlled by dropdown list and range slider
- Creates pie and scatter chart with callbacks for connection in conjunction with cloud server
- GitHub URL:
<https://github.com/jaustrylle/applied-data-science-capstone/blob/main/spacex-dash-app.py>



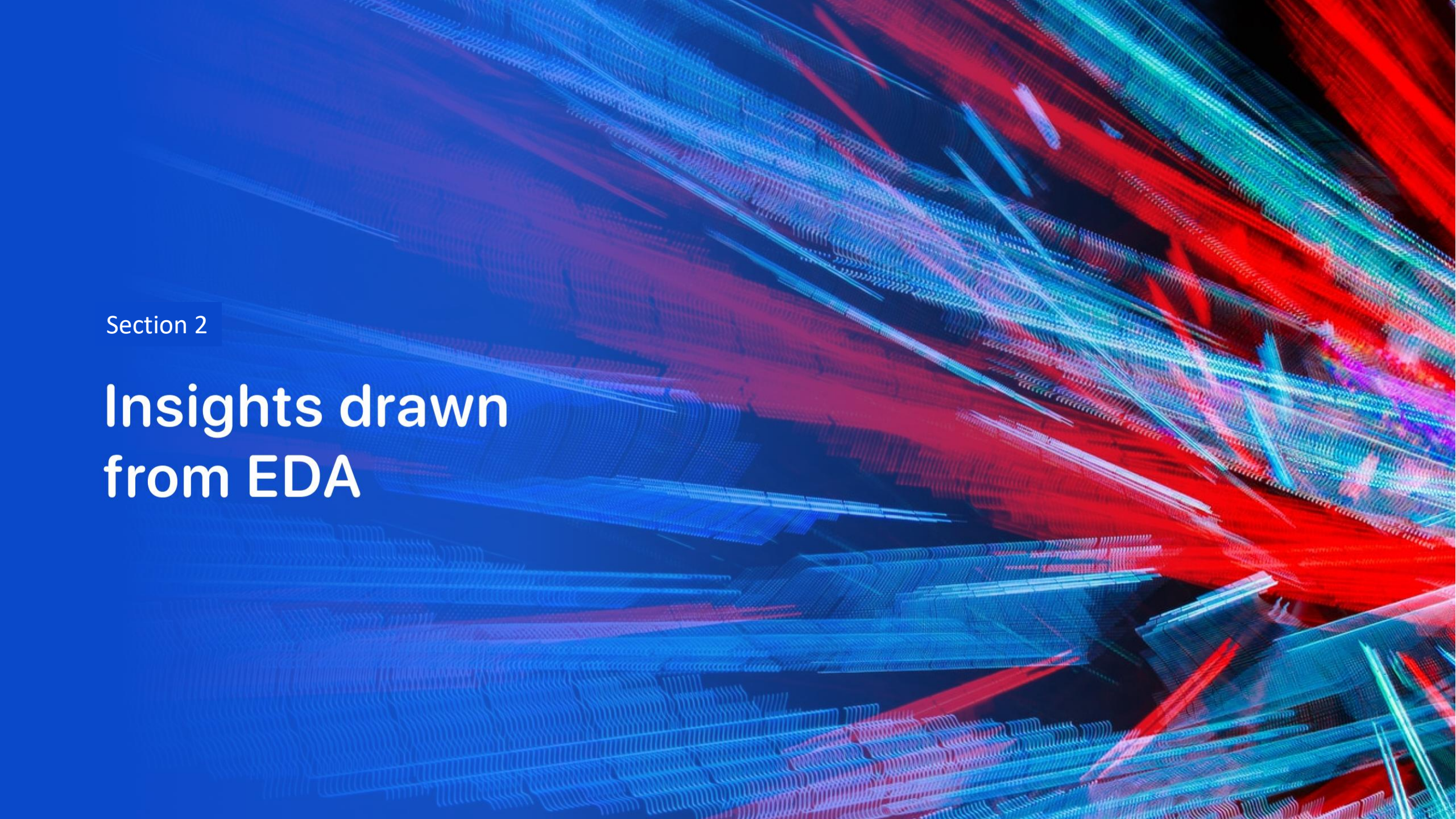
Predictive Analysis (Classification)

- Features were transformed via Standard Scaler; Grid Search performed for parameters
- Model accuracies on test data:
 - Logistic Regression – 83.33%
 - SVM – 83.33%
 - Decision Tree – 83.33%
 - KNN – 83.33%
- GitHub URL:
https://github.com/jaustrylle/applied-data-science-capstone/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb



Results

- Exploratory data analysis results
 - Chronological optimization from volatile early flights to high-reliability state after Flight 40
 - Payload core correlations under 5,000 kg threshold
 - Orbital success profiles varied due to highly energetic GTO yielding a low 52% success rate
- Interactive analytics demo in screenshots
 - Global site distributions seen in pie chart
 - Dynamic parameter filters seen in Folium
- Predictive analysis results
 - Steady 83.33% across model types in Jupyter Notebook

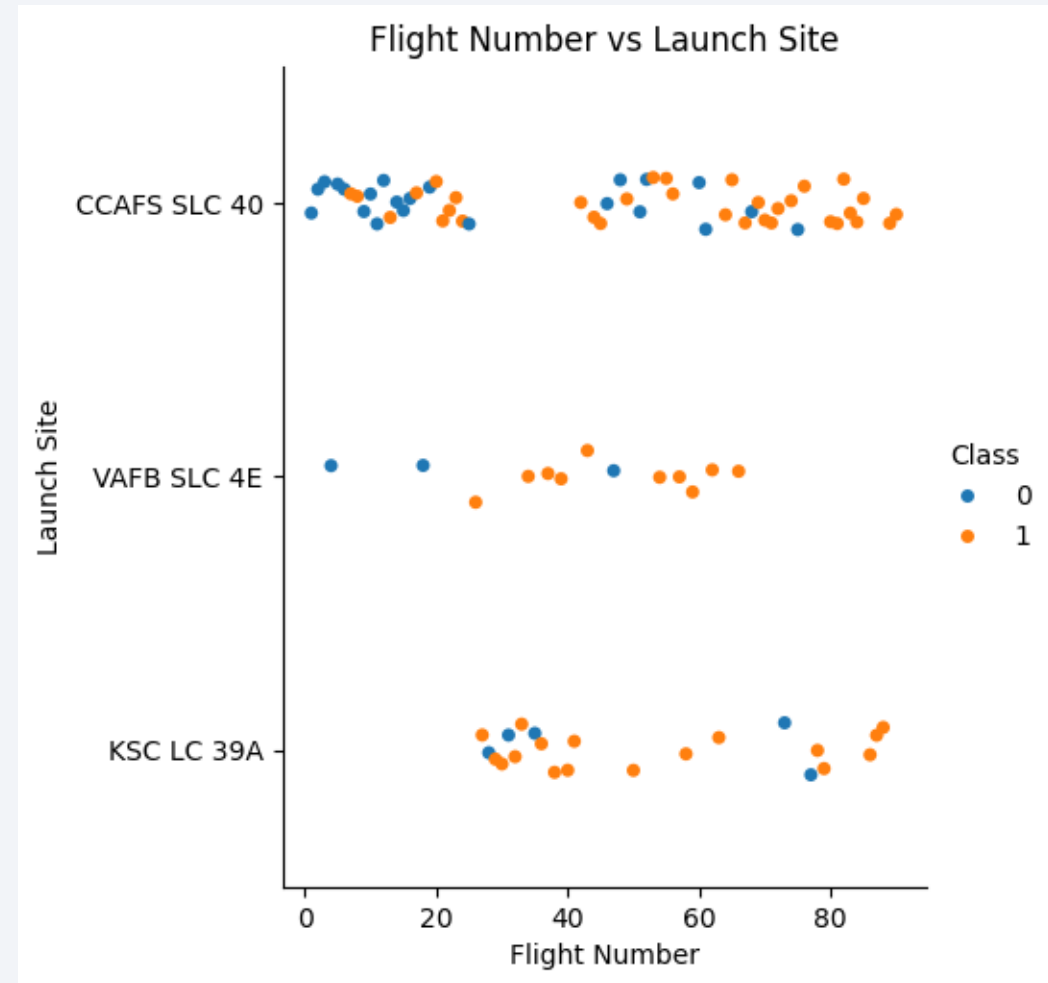
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower half of the image. The overall effect is high-tech and digital.

Section 2

Insights drawn from EDA

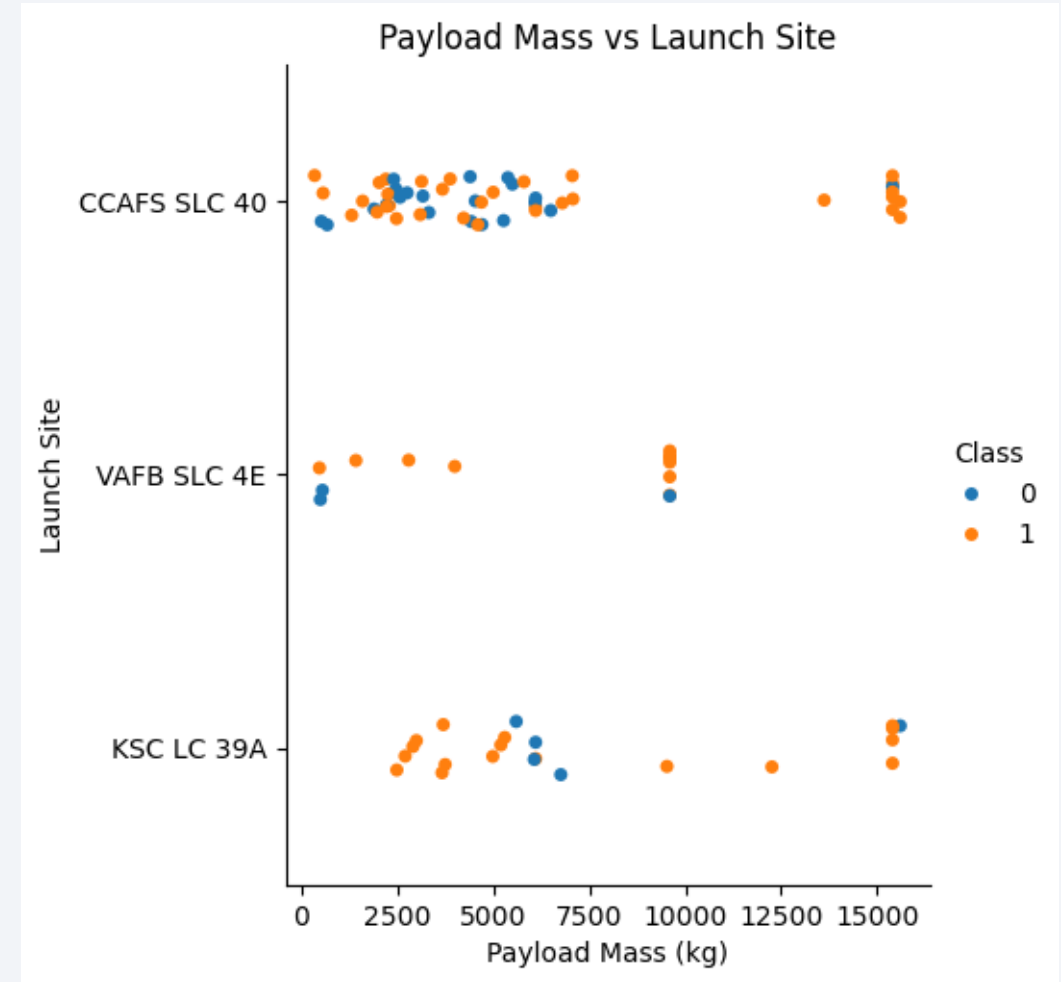
Flight Number vs. Launch Site

- CCAFS SLC 40 was used heavily early in launch testing
- VAFB SLC 4E was rarely used early on and gradually faded out of usage
- Success rate of later flights improved



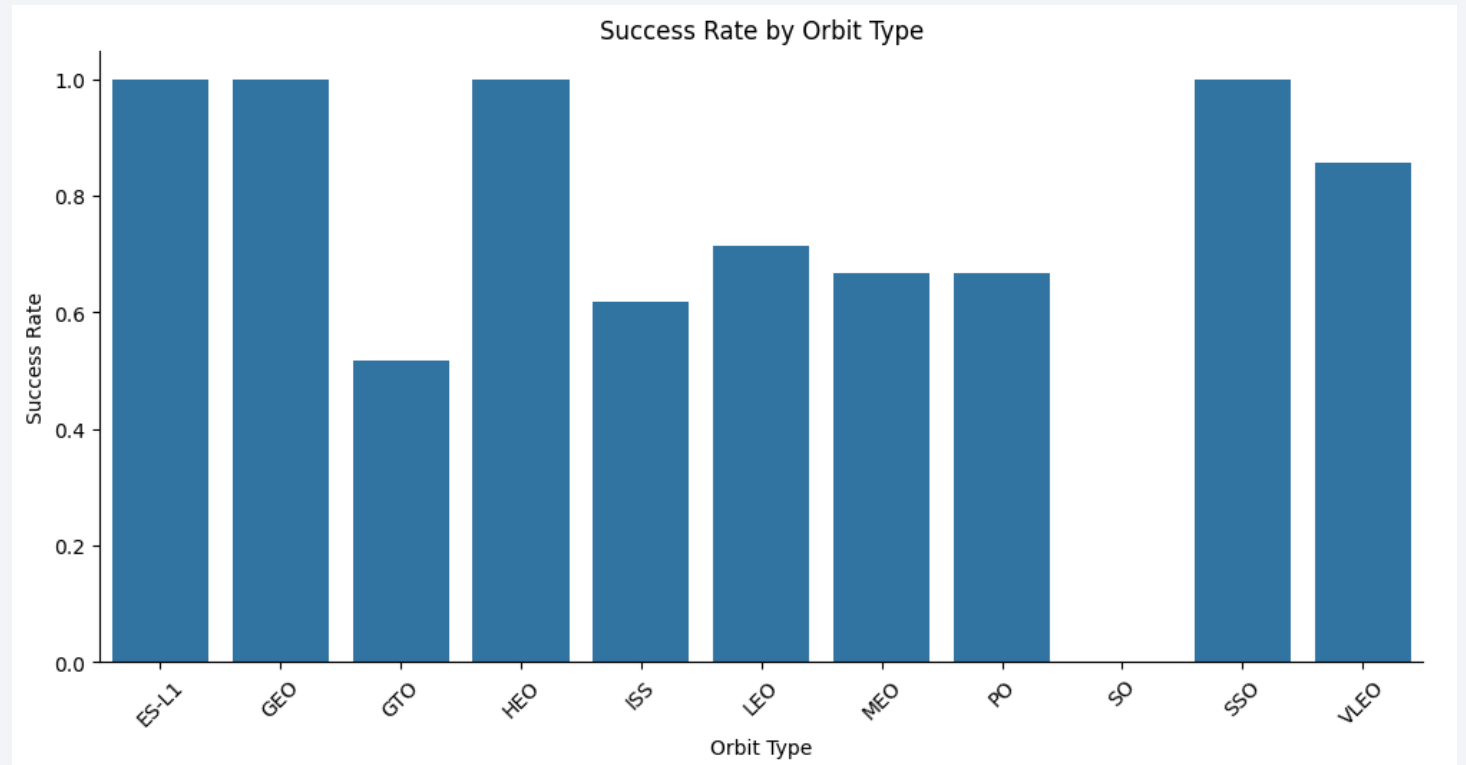
Payload vs. Launch Site

- VAFB SLC 4E handles specific payload profiles
- KSC LC 39A handles medium to heavy payload
- CCAFS SLC 40 handles diverse payloads
- Heavy payloads succeed more often than light payloads



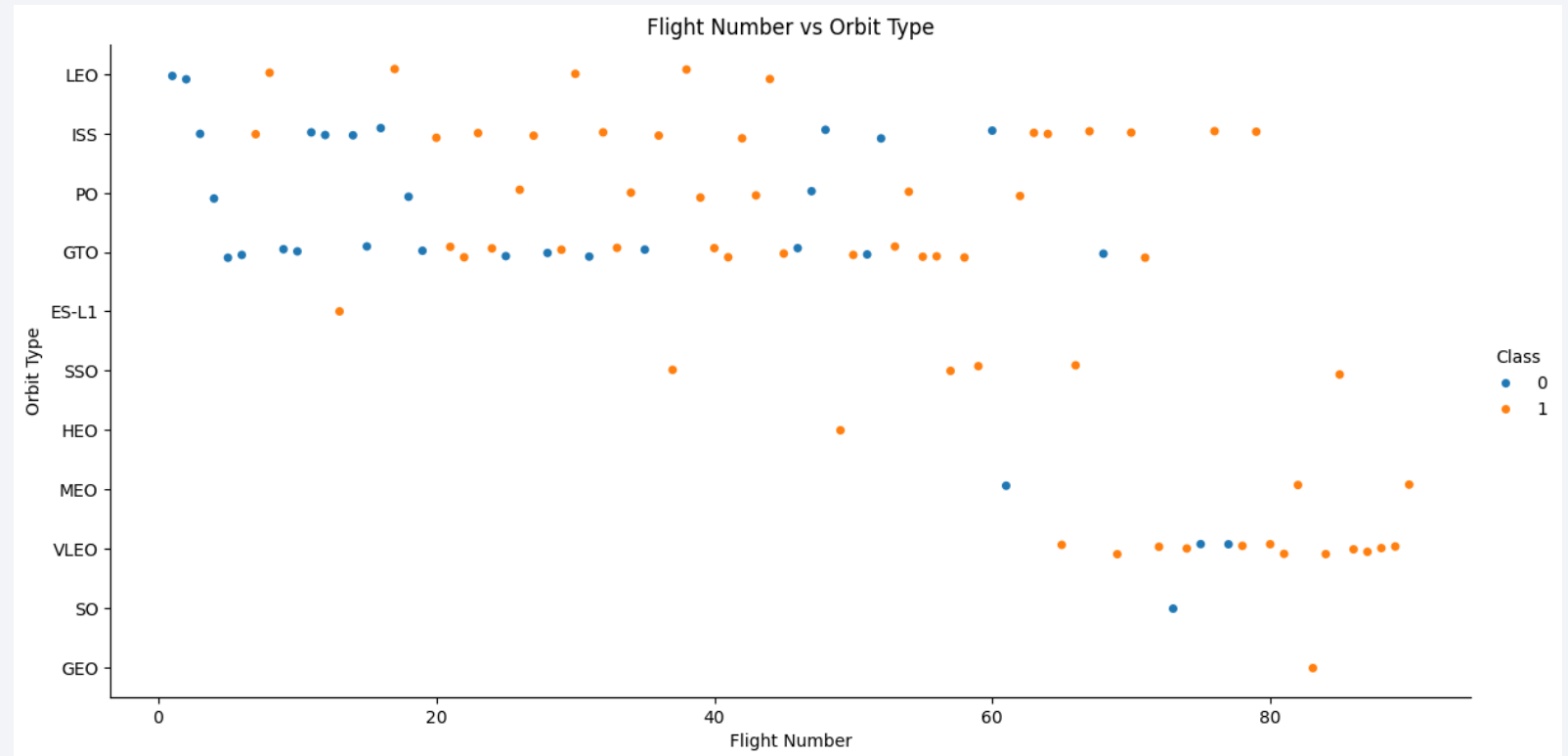
Success Rate vs. Orbit Type

- Underperforming:
 - GTO (due to high energy requirement)
 - SO (likely no tested missions)
- Perfect success rates:
 - ES-L1, GEO, HEO, SSO
 - Could be result of lower sample size



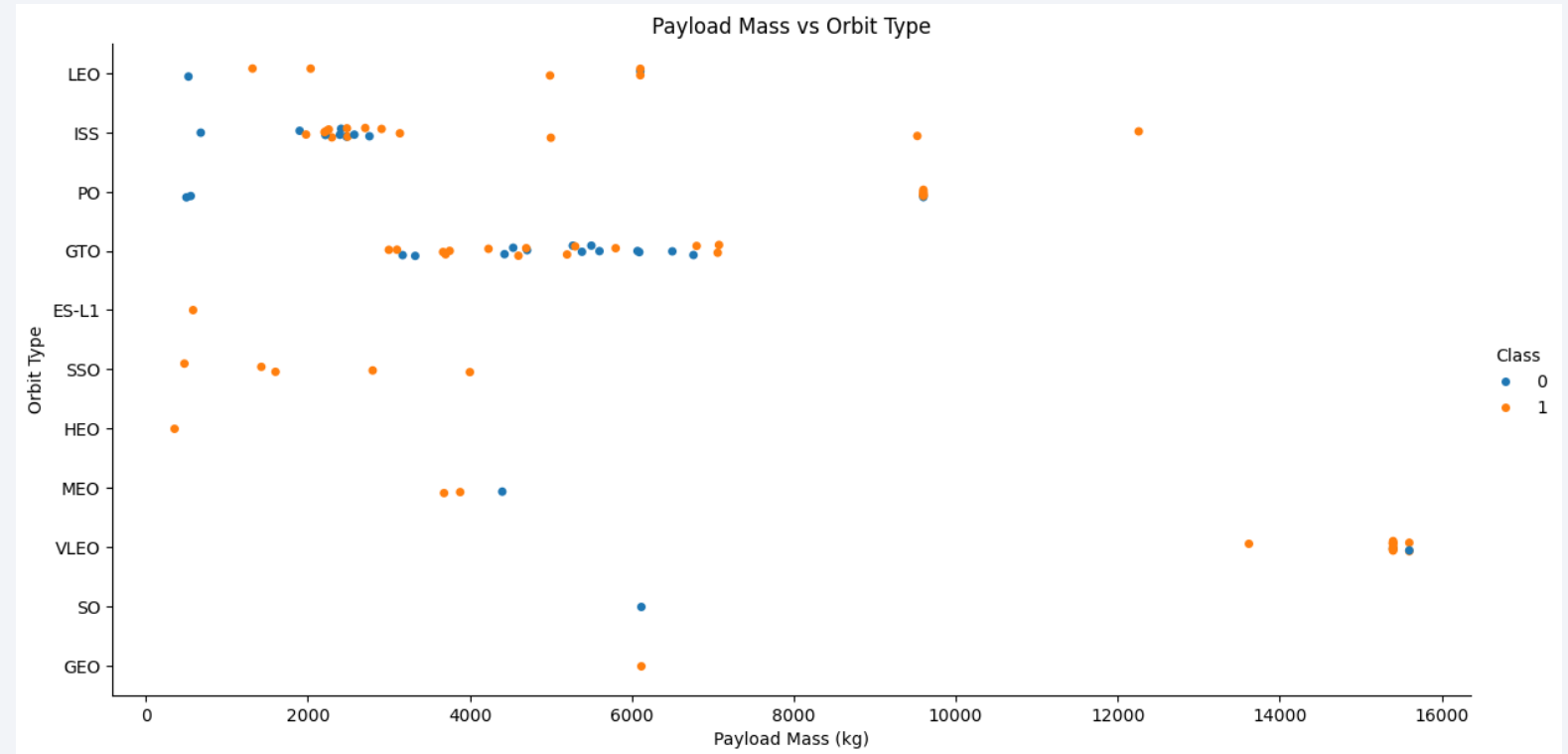
Flight Number vs. Orbit Type

- Early phase – Flights 0-30
 - GTO/LEO/ISS targeted
- Mid phase – Flights 30-60
 - More varied and complex orbits
- Late phase – Flights 60-90
 - VLEO targeted (due to mass production)



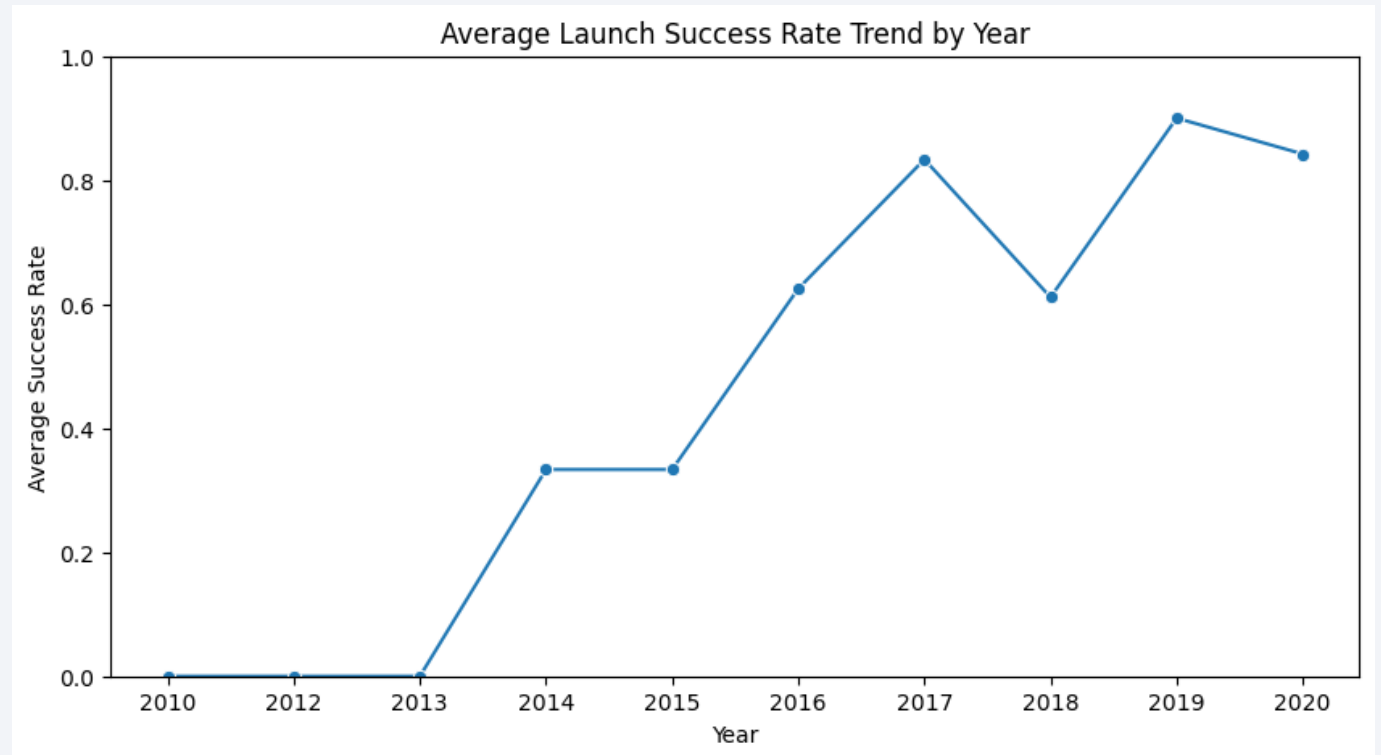
Payload vs. Orbit Type

- VLEO handle the heaviest payloads and are overwhelmingly successful
- GTO handles highly-defined payloads (2,500-7,000 kg)
- ISS handles 1,500-3,500 kg
- SSO/HEO mostly under 4,000 kg



Launch Success Yearly Trend

- 2010-13 – average success rate flat at 0%
- 2014-15 – reaches 33.3% and stabilizes for 2 years
- 2016-17 – steep upward trajectory starts; peak at 83% success rate
- 2018 – drop in trend line to 61%
- 2019-20 – success rate rebounds to 90% then huddles at 84%



All Launch Site Names

%%sql

```
SELECT DISTINCT Launch_Site  
FROM SPACEXTABLE;
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

```
%%sql
```

```
SELECT *
```

```
FROM SPACEXTABLE
```

```
WHERE Launch_Site LIKE 'CCA%'
```

```
LIMIT 5;
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

```
%%sql  
SELECT  
SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass  
FROM SPACEXTABLE  
WHERE Customer LIKE '%NASA (CRS)%';
```

Total_Payload_Mass
48213

Average Payload Mass by F9 v1.1

```
%%sql  
SELECT  
AVG(PAYLOAD_MASS__KG_) AS Average_Payload_Mass  
FROM SPACEXTABLE  
WHERE Booster_Version = 'F9 v1.1';
```

Average_Payload_Mass
2928.4

First Successful Ground Landing Date

%%sql

SELECT

MIN(Date) AS First_Successful_Ground_Pad_Landing_Date

FROM SPACEXTABLE

WHERE Landing_Outcome = 'Success (ground pad)';

First_Successful_Ground_Pad_Landing_Date
--

2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

```
%%sql  
SELECT  
Booster_Version  
FROM SPACEXTABLE  
WHERE Landing_Outcome = 'Success (drone ship)'  
AND PAYLOAD_MASS__KG_ > 4000  
AND PAYLOAD_MASS__KG_ < 6000;
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

%%sql

```
SELECT Mission_Outcome, COUNT(*) AS Total_Count  
FROM SPACEXTABLE  
GROUP BY Mission_Outcome;
```

Mission_Outcome	Total_Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

```
%%sql  
SELECT Booster_Version  
FROM SPACEXTABLE  
WHERE Payload_Mass__Kg_ = (  
SELECT MAX(Payload_Mass__kg_)  
FROM SPACEXTABLE  
);
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

```
%%sql
SELECT
substr(Date, 6, 2) AS Month,
Landing_Outcome,
Booster_Version,
Launch_Site
FROM SPACEXTABLE
WHERE substr(Date, 0, 5) = '2015'
AND Landing_Outcome LIKE '%Failure%'
AND Landing_Outcome LIKE '%drone ship%';
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

%%sql

SELECT

Landing_Outcome,

COUNT(*) AS Outcome_Count

FROM SPACEXTABLE

WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'

GROUP BY Landing_Outcome

ORDER BY Outcome_Count DESC;

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

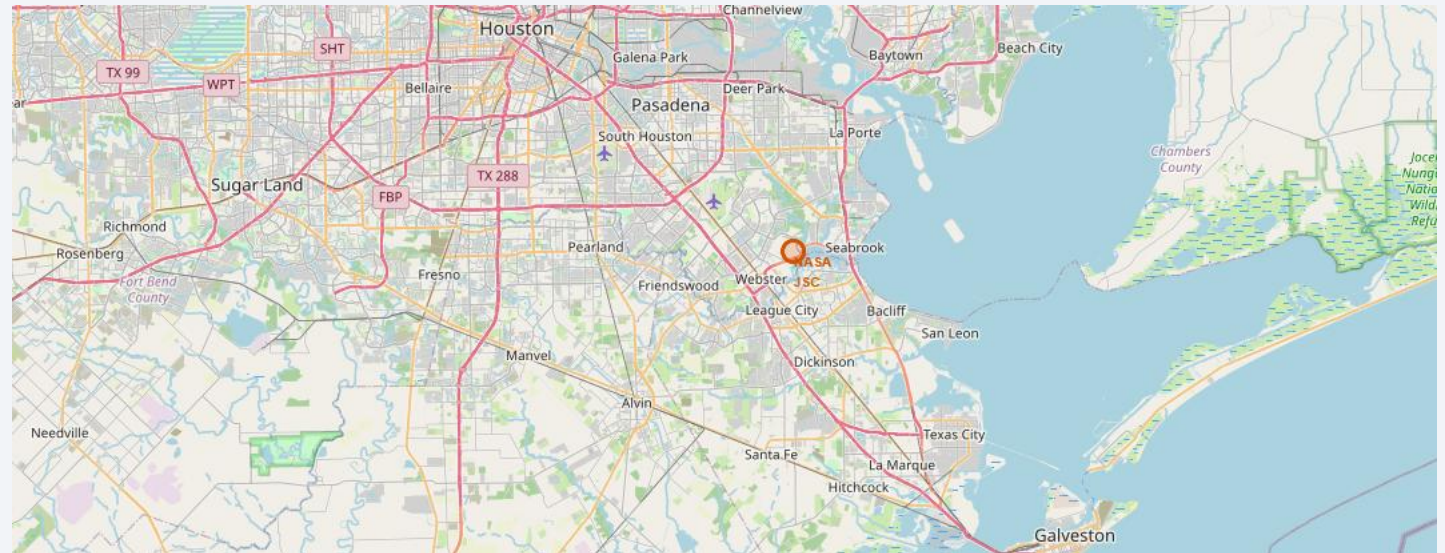
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

Launch Sites Proximities Analysis

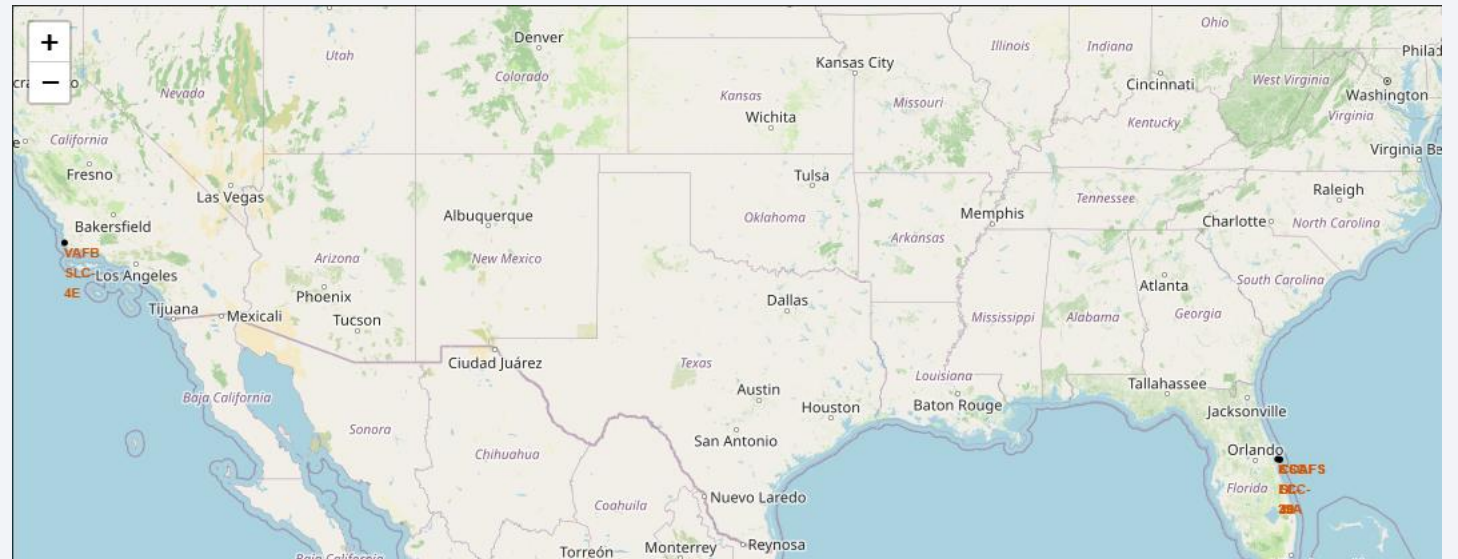
Folium Map – NASA Johnson Space Center

- Blue circle shows space center's coordinate with popup label
- Icon functions as text label



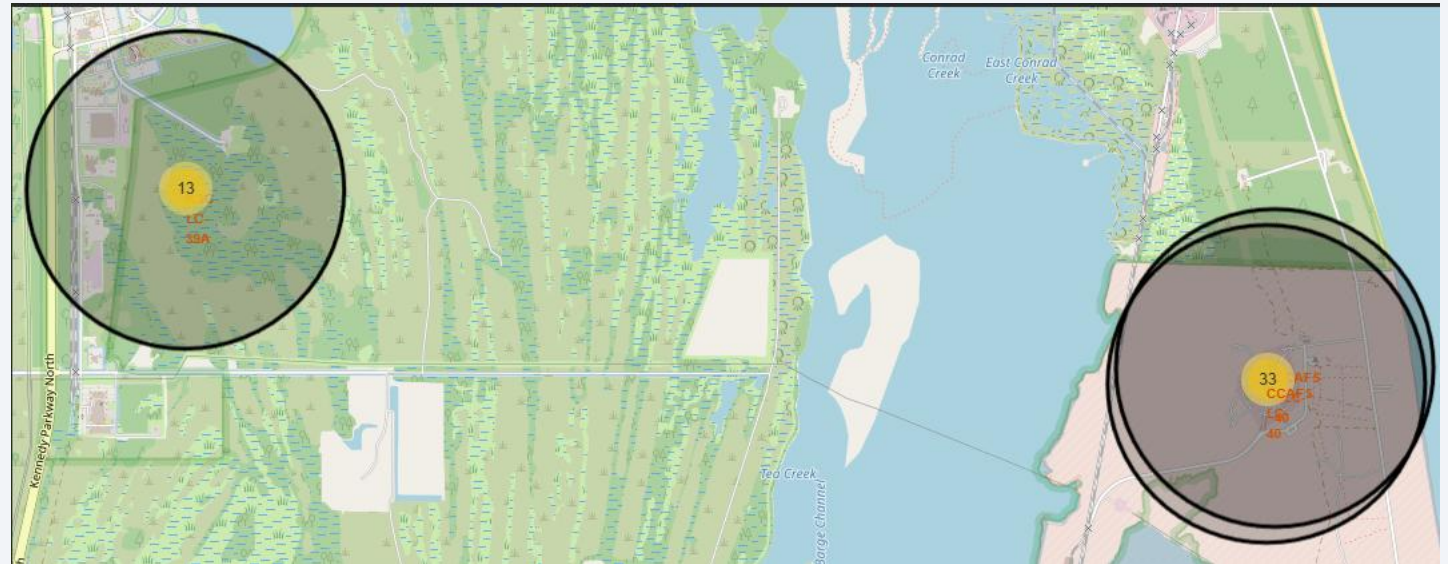
Folium Map – All Launch Sites

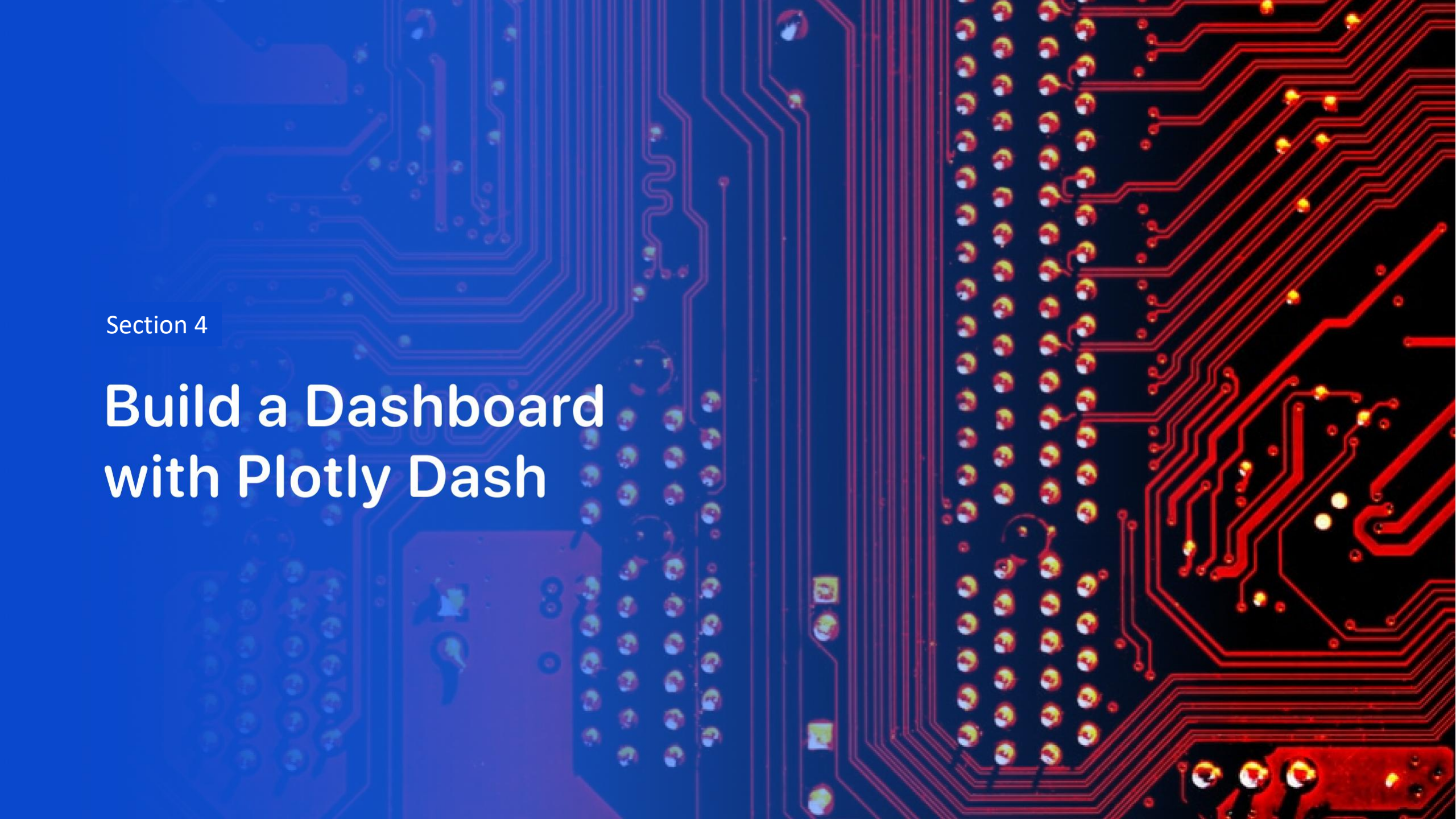
- Plotted all launch sites from dataset using iterative loop for map markers and coordinates



<Folium Map Screenshot 3>

- Added marker clusters for each launch result to indicate whether each launch is classified as “fail” or “success”



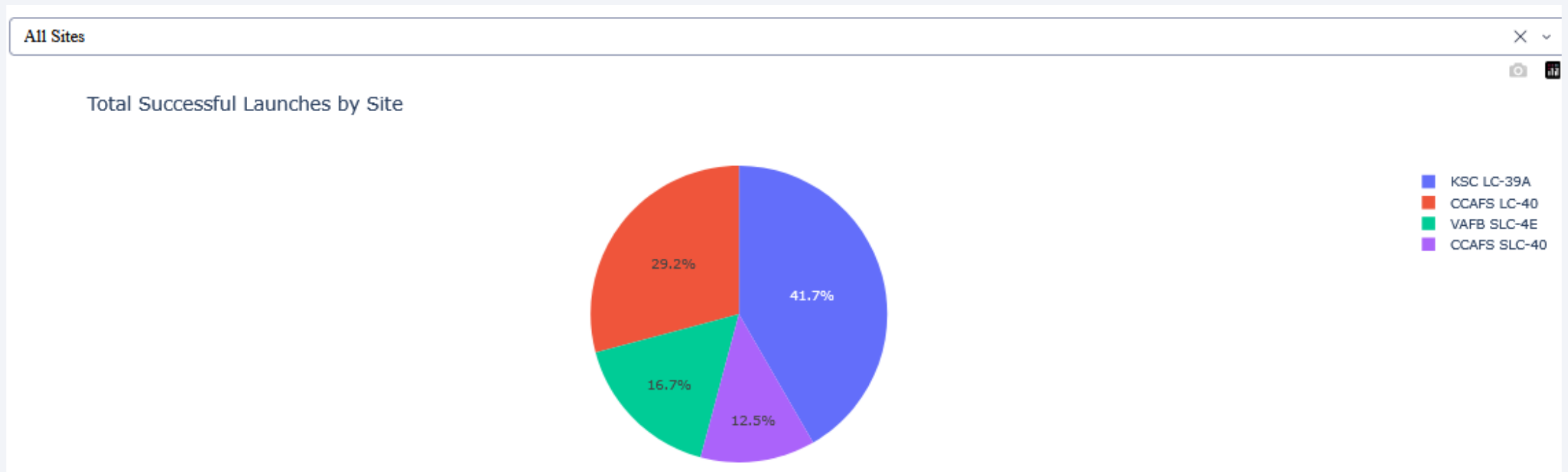


Section 4

Build a Dashboard with Plotly Dash

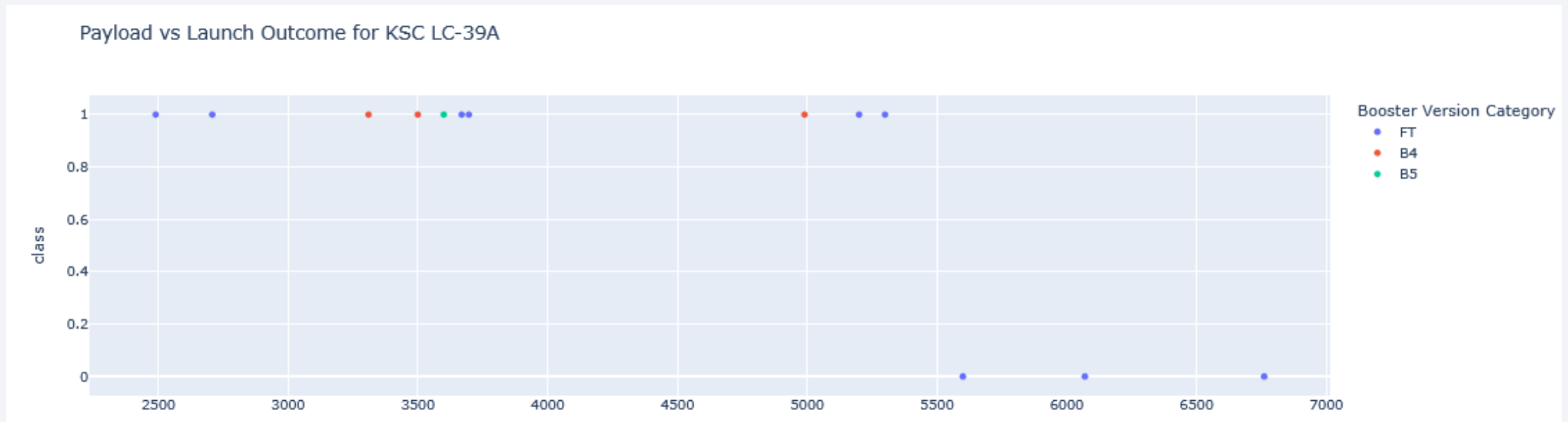
Dashboard – Pie Chart

- Kennedy Space Center (LC-39A) holds largest individual share of successful launches (41.7%)
- Vandenberg Air Force Base (SLC-4E) holds smallest individual share (16.7%)



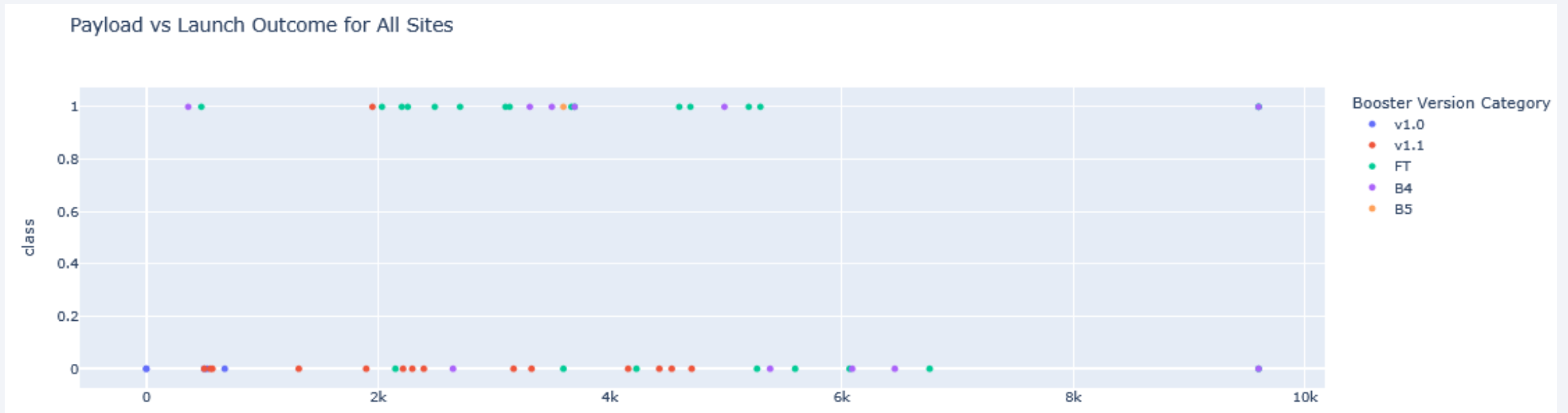
Dashboard – Scatter Chart by Booster Class

- Failure cluster – 5,500-7,000 kg
 - All 3 belonged to Full Thrust (FT)
- Success cluster – 2,500-5,300 kg
 - Mix of all three booster versions



Dashboard – All Sites by Booster Version

- Early (v1.0 & v1.1) – almost exclusively on line of failure
- Intermediate (FT & B4) – both succeed and fail
- Late (B5) – clustered around success line

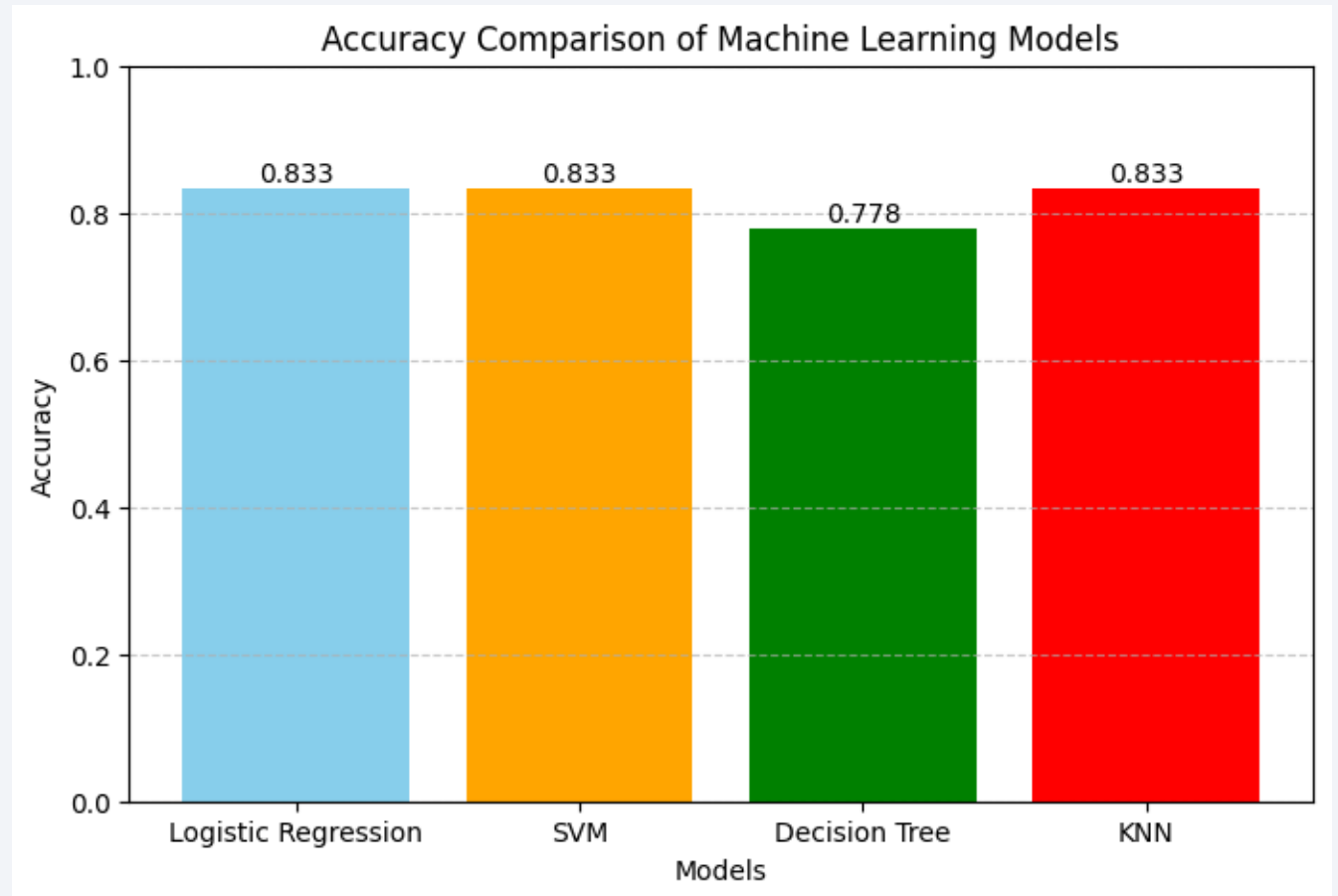


Section 5

Predictive Analysis (Classification)

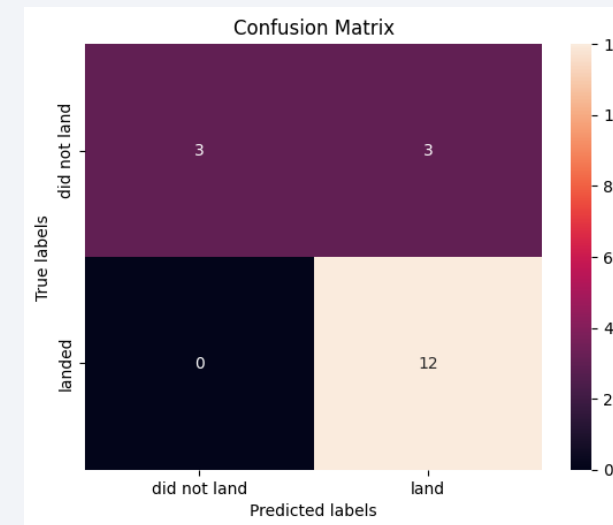
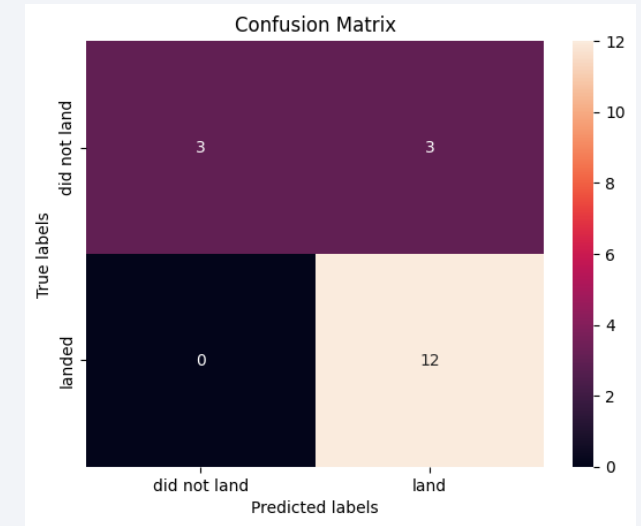
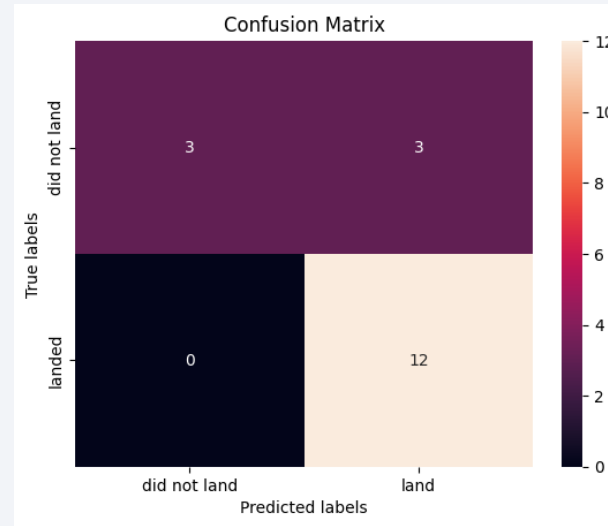
Classification Accuracy

- All of the 4 models have similar accuracy
- When Decision Tree model was tested a second time in Google Collab instead, accuracy dropped to 77.8%



Confusion Matrix

- KNN, Logistic Regression, and SVM performed equally well at around 83.3%
- Decision Tree model performed at first at 83.3%, but after the model ran a 2nd time, it performed at 77.8% due to random state



Conclusions

- More work to be done on Folium maps for real-time analytics and improved mapping
- More iterations mitigate risk of launch failure due to hardware optimization, flight experience, and design maturity
- Facilities on the East Coast currently serve as centers of high-frequency launch cadence (heaviest and most frequent payloads)
- Booster iterations differentiates between success and failure, pushing operational envelopes to structural failure if beyond 5,500 kg

Appendix

- Extra code snippets to mitigate API error after initially fetching data:

```
# Takes the dataset and uses the rocket column to call the API and append the data to the list
def getBoosterVersion(data):
    rocket_map = {
        "5e9d0d95eda69955f709d1eb": "Falcon 1",
        "5e9d0d95eda69973a809d1ec": "Falcon 9",
        "5e9d0d95eda69970e809d1ec": "Falcon Heavy"
    }

    for x in data['rocket']:
        BoosterVersion.append(rocket_map.get(x))
```

```
# Takes the dataset and uses the payloads column to call the API and append the data to the lists
def getPayloadData(data):
    for load in data['payloads']:
        # Static values are not needed for the lab output
        PayloadMass.append(None)
        Orbit.append(None)
```

```
# Takes the dataset and uses the launchpad column to call the API and append the data to the list
def getLaunchSite(data):
    launchpad_map = {
        "5e9e4502f5090995de566f86": ["Kwajalein Atoll", 167.7431292, 9.047721],
        "5e9e4501f509094ba4566f84": ["CCSFS SLC 40", -80.577366, 28.5618571],
        "5e9e4501f509094ba4566f85": ["VAFB SLC 4E", -120.610829, 34.632093]
    }

    for x in data['launchpad']:
        if x in launchpad_map:
            LaunchSite.append(launchpad_map[x][0])
            Longitude.append(launchpad_map[x][1])
            Latitude.append(launchpad_map[x][2])
        else:
            LaunchSite.append(None)
            Longitude.append(None)
            Latitude.append(None)
```

```
# Takes the dataset and uses the cores column to call the API and append the data to the lists
def getCoreData(data):
    for core in data['cores']:
        Outcome.append(str(core['landing_success']) + " " + str(core['landing_type']))
        Flights.append(core['flight'])
        GridFins.append(core['gridfins'])
        Reused.append(core['reused'])
        Legs.append(core['legs'])
        LandingPad.append(core['landpad'])

    Block.append(None)
    ReusedCount.append(None)
    Serial.append(None)
```

Thank you!

